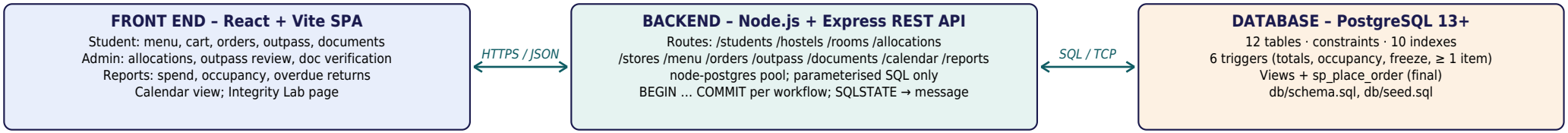
CampusConnect — Application Design

DBMS Course Project · Mid-Sem Submission · Proposed workflows, technology stack and high-level architecture · Team: Group-15 – Shubham Sagar (2024A3PS0394H), Jay Kimtani (2024A3PS0392H), Sai Gautam Reddy (2024A3PS0429H), Krishna Gupta (2024A3PS0380H), Sreejith Gundla (2024A3PS0427H)

1. High-level architecture



The system follows Front End ↔ Application/Backend ↔ Relational Database. The backend is a thin layer: it maps HTTP requests to SQL, wraps multi-statement workflows in transactions and returns database errors as readable messages. **Business rules are not re-implemented in the API** – they live in constraints and triggers (see Database Design Document).

2. Technology stack

Layer	Choice	Why
Database	PostgreSQL 13+ (psql, pgAdmin)	Partial unique indexes, deferrable constraint triggers, PL/pgSQL – all used by the schema
Backend	Node.js 20 + Express, node-postgres (pg) pool	Direct SQL (no ORM) so queries, transactions and error codes stay visible for the viva
Front end	React 18 + Vite, fetch API, plain CSS	Simple SPA; role switch (Student / Admin) in the UI
Tooling	Git + GitHub, `.env` for DB credentials, seed-data script (Python or SQL)	Reproducible setup: schema → seed → run

3. End-to-end workflows (each exercises a different part of the database)

#	Workflow	Steps through the app	Database behaviour demonstrated
W1	**Place / cancel an order** (transaction, coordinated multi-table change)	Student picks a store → sees available menu items → builds a cart → submits. Later cancels the order (or reinstates it).	BEGIN; INSERT orders; INSERT order_item × n (unit_price copied from menu_item); COMMIT. trg_item recomputes ORDERS.total_amount, trg_order adds it to STUDENT.total_deductions_amount; the deferred trigger rejects an empty order at COMMIT; cancel subtracts the total again and freezes the items.
W2	**Outpass lifecycle and reporting** (search, filter, aggregation)	Student requests an outpass (reason, out_at, return_by) → admin sees the pending queue → approves / rejects → student's return is recorded → admin opens the overdue and approval-rate reports.	Multi-table joins (outpass × student × admin × allocation); filter on idx_outpass_status; GROUP BY admin / month; correlated query for overdue returns (approved, returned_at IS NULL, return_by < now()).
W3	**Room allocation and Integrity Lab** (constraint enforcement)	Admin allocates or transfers a student (end the current stay, start a new one in one transaction) → occupancy table updates. An Integrity Lab page fires invalid operations and shows the error.	trg_alloc_occupancy keeps ROOM.occupancy exact; rejected operations: second ACTIVE allocation (uq_one_active_allocation), outpass with return_by ≤ out_at (chk_outpass_window), item from another store (composite FK), editing items of a cancelled order (trg_item_frozen), order without items (deferred trigger).

4. Planned SQL reports (≥ 5 non-trivial queries for the final submission)

#	Report	Technique
Q1	Monthly revenue and order count per store (excluding cancelled orders)	JOIN orders–campus_store, GROUP BY store, month; HAVING
Q2	Top 3 best-selling items in every store	CTE over order_item + RANK() OVER (PARTITION BY store_name …)
Q3	Hostel occupancy: rooms, occupied rooms and students per hostel	JOIN hostel–room–room_allocation; aggregate; LEFT JOIN for empty hostels
Q4	Students whose approved outpass is overdue, with reviewing admin and hostel	Multi-table JOIN + correlated subquery / NOT EXISTS on returned_at
Q5	Students who have ordered from every campus store (relational division)	Double NOT EXISTS
Q6	Students whose total spend is above the average spend of their hostel	Subquery with aggregate per hostel; compares against stored total_deductions_amount
Q7	Audit queries: stored totals and occupancy equal recomputed values	LEFT JOIN + GROUP BY … HAVING (must return zero rows)

5. Screens (front end)

Role	Screens
Student	Dashboard (room, spend, next calendar events) · Stores and menu · Cart and my orders · Outpass request / history · My documents
Admin	Hostels and rooms · Allocate / transfer · Outpass queue (approve / reject) · Document verification · Stores and menu management · Calendar editor
Common	Reports page (Q1–Q6) · Integrity Lab (named failures shown with the PostgreSQL error)

6. REST API outline

Resource	Endpoints (method path)	Touches
Orders	POST /orders (body: student_id, store_name, items[]) · PATCH /orders/:id/status · GET /students/:id/orders	orders, order_item, student (via triggers)
Outpass	POST /outpass · PATCH /outpass/:id/review · PATCH /outpass/:id/return · GET /outpass?status=	outpass_request, admin, student
Allocation	POST /allocations (transfer in one transaction) · PATCH /allocations/:key/end · GET /hostels/:name/occupancy	room_allocation, room (via trigger)
Master data	CRUD /students · /hostels · /rooms · /stores · /menu · /documents · /calendar	all remaining tables
Reports / lab	GET /reports/q1 … q6 · POST /lab/:case (runs the invalid statement in a rolled-back transaction)	views and queries

7. Plan to the final submission (15-Nov)

Milestone	Deliverable
Mid-sem (11-Oct)	Schema with constraints, triggers and indexes; ER diagram; design documents; GitHub repo with README
Next	Seed script (≥ 500 records across all tables) · views and sp_place_order · queries Q1–Q7 · REST API skeleton
Then	React screens for W1–W3 · Integrity Lab · testing of every trigger / constraint with the audit queries
Final	Complete repository (app, DB scripts, README, final design PDF) · rehearsal of the live demo and viva